2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
October 23-27, 2022, Kyoto, Japan

Locomotion Policy Guided Traversability Learning using Volumetric Representations of Complex Environments

Jonas Frey, David Hoeller, Shehryar Khattak, Marco Hutter

***Abstract*— Despite the progress in legged robotic locomotion, autonomous navigation in unknown environments remains an open problem. Ideally, the navigation system utilizes the full potential of the robots' locomotion capabilities while operating within safety limits under uncertainty. The robot must sense and analyze the traversability of the surrounding terrain, which depends on the hardware, locomotion control, and terrain properties. It may contain information about the risk, energy, or time consumption needed to traverse the terrain. To avoid hand-crafted traversability cost functions we propose to collect traversability information about the robot and locomotion policy by simulating the traversal over randomly generated terrains using a physics simulator. Thousand of robots are simulated in parallel controlled by the same locomotion policy used in reality to acquire 57 years of real-world locomotion experience equivalent. For deployment on the real robot, a sparse convolutional network is trained to predict the simulated traversability cost, which is tailored to the deployed locomotion policy, from an entirely geometric representation of the environment in the form of a 3D voxel-occupancy map. This representation avoids the need for commonly used elevation maps, which are error-prone in the presence of overhanging obstacles and multi-floor or low-ceiling scenarios. The effectiveness of the proposed traversability prediction network is demonstrated for path planning for the legged robot ANYmal in various indoor and natural environments.**

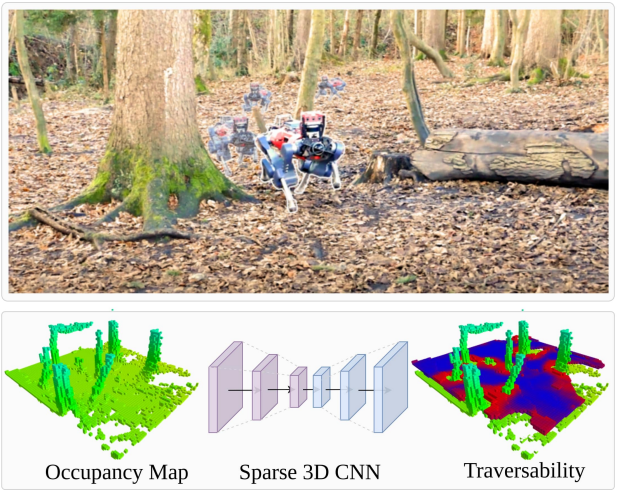


Fig. 1. The legged robot ANYmal is deployed in a forest. A robot-centric occupancy map is generated using two onboard LiDAR sensors. Our real-time capable proposed sparse 3D neural network predicts the traversability of the surrounding terrain in 3D, tailored to the deployed locomotion policy. Blue indicates risk-free traversable, while red indicates traversable under high-risk.

I. INTRODUCTION

Humans and animals have the remarkable ability to estimate where they can go safely and efficiently. The capability to quickly assess one's surrounding is an essential selection criterion for survival in a variety of scenarios e.g., when a mouse has to escape from a cat into a narrow hole in the wall. Similar constraint-aware terrain assessment capabilities are desirable for autonomous mobile robot deployment in complex unstructured environments. This assessment of the surrounding terrain can be used to navigate safely from a start to a goal position, during exploration, or to build a better semantic understanding of the scene. A specific terrain is associated with a cost, which depends on the capability of the robotic system, locomotion policy, and the structure of the deployment environment. The traversability assessment is performed based on multiple application-dependent objectives, which may include: Desired goal position, time and risk minimization, collision avoidance, safety margins, or energy efficiency [22].

To acquire information about its surrounding, a robot is equipped with a suite of exteroceptive sensors. Typically, geometrical information about the environment is accumulated in a map representation by fusing depth measurements from LiDARs or depth cameras. For this, ground robots typically utilize a 2.5D elevation map representation due to its computational efficiency, in which each cell within a regularly sampled grid at a fixed resolution stores the estimated height value [5]. During path planning for ground robots the elevation map can be used to restrict the search space, given that the base height of the robot is constrained by the underlying terrain geometry. This reduces the planning time given that not all poses in 3D space have to be considered for path planning.

However, this 2.5D elevation map representation cannot represent multi-floor environments adequately [14], as without additional semantic understanding, integration of depth measurements coming from over-hang obstacles or the ceiling corrupts the elevation map and leads to sub-optimal path planning. Transitioning to a more expressive 3D voxel occupancy representation [20] of the environment resolves the aforementioned problems, but requires new methods for efficient traversability assessment. To this end, this work proposes a robot-centric traversability estimation

J. Frey, D. Hoeller, S. Khattak and M. Hutter are with the Department of Mechanical and Process Engineering, ETH Zürich, 8092 Zürich, Switzerland {jonfrey, skhattak, dhoeller, mahutter}@ethz.ch. D. Hoeller is also associated with NVIDIA.This work was supported by the Swiss National Science Foundation (SNSF) through project 166232, 188596, the National Centre of Competence in Research Robotics (NCCR Robotics), and the European Union's Horizon 2020 research and innovation program under grant agreement No.780883.

2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) | 978-1-6654-7927-1/22/$31.00 ©2022 IEEE | DOI: 10.1109/IROS47612.2022.9982190

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 14:53:34 UTC from IEEE Xplore. Restrictions apply.

module, which predicts the traversability of the terrain, while being aware of the robots' specific locomotion capabilities. As an input, the volumetric occupancy representation is provided, which captures geometric information sufficient to avoid positive obstacles, negative obstacles, and steep slopes. To exploit the sparsity of the underlying problem, we use a sparse 3D convolutional neural network (CNN) to estimate the terrain traversability in real-time. The network is trained on data gathered within a highly-efficient physics simulator, where the success rate of executing different local motion commands on randomly generated terrain is evaluated inspired by the work of [2], [8], [30]. We deploy the same locomotion policy in simulation as on the real robots to achieve successful sim-to-real transfer. The main contributions of the presented work are:

- Predicting terrain traversability using volumetric environment representations to allow for motion planning in complex environments with over-hanging obstacles.
- Application of a sparse 3D CNN trained exclusively on simulated data for traversability estimation.
- Creating a framework for large-scale traversability experience collection within a physics simulator.
- Deployment in real complex and natural environments and showcasing path planning application for ANYmal.

II. RELATED WORK

In this section, we provide an overview of traversability estimation methods deployed for ground robots. We categorize the approaches based on their input representation into visual and geometrical methods. Additionally, we provide an overview of the most relevant work for deep learning on volumetric voxel data.

A. Visual Traversability Estimation

Images contain rich geometrical and semantic information about the environment. Teams within the Darpa LAGR project [12] deployed self-supervised learning approaches to classify from images the affordance necessary to traverse with wheeled robots outdoor. To overcome the issue of limited supervision signal Otsu et al. [21] use a support vector machine trained in a self-supervised fashion to classify terrain for a mars rover. The supervision signal is extracted from vibration data experienced by the robot and associated in hindsight with the previously captured images of the terrain. Wellhausen et al. [27] extend this concept for legged robots and train a CNN to estimate terrain properties with a supervision signal acquired by force-torque sensors in the robot's feet. In conclusion visual data contains rich information to estimate terrain friction, terrain roughness, foothold quality, vegetation, support surfaces, and obstacles. Extraction of this information under sensor noise, motion blur, bad lighting conditions remains a challenging problem and collecting labeled high-quality data for supervised learning methods is expensive. Further, using synthetic camera data is often prohibited by the sim-to-real gap induced by the mismatch between the simulated camera data and data encountered by a robot during its mission [6].

B. Geometrical Traversability Estimation

Apart from cameras, most mobile robots are equipped with LiDAR and depth cameras to acquire geometrical information. This data is less rich in semantic information but easier to process and simulate. Individual depth measurements can be used for precise localization and fused into different representations including pointclouds, 2.5D elevation maps [5], [17] and 3D voxel-based occupancy maps. The most common frameworks to create the latter are Voxblox [20], Octomap [11], Fiesta [10] or implementations build around the VDB representation [19].

Wermerlinger et al. [29] use an elevation map and identify the traversability for a legged robot based on hand-crafted heuristics such as slopes, terrain roughness, and steps. Different risk sources are assessed from elevation data [1], [4]. The individual risks are stored in a Multi-Layer Traversability Map (MLT), which by superposition of all layers, is used to estimate the total risk for planning. Wellhausen et al. [28] train a CNN to estimate good footholds for a legged robot from an elevation map. The training dataset of good footholds is hand-labeled.

In contrast to previous work, Kruesi et al. [14] use pointclouds to estimate the static terrain traversability for a wheeled robot to achieve safe motion planning. Methods such as GBPlanner2 [15] illustrate the benefit of a volumetric environment representation for exploration and global path planning. The planner is primarily developed for aerial robots, but when adapted to legged systems, the ground surface estimation is solved based on heuristics and traversability not considered for planning. All of the previous geometrical methods rely on hand-crafted heuristics and human expertise to evaluate traversability.

In contrast, Chavez-Garcia et al. [2] simulate a wheeled robot executing local motion commands and gather traversability information in simulation. They train a CNN to classify the traversability of elevation maps. Guzzi et al. [8] extend this concept for legged robots and, additionally to the robot-centric elevation map provide a relative target pose to estimate the time and probability of reaching the target. Lastly, Yang et al. [30] follow a similar procedure to simulate the traversability of a given terrain for a legged robot. Information about the time, risk, and energy needed for a robot to execute a local motion command is collected in simulation. A CNN network is trained using this data and predicts the collected quantities similar to [8]. Our approach follows the idea of [8] to simulate the robot to acquire locomotion policy-guided traversability information. All previous work rely on elevation maps, which are error-prone in multi-floor or overhanging obstacles scenario, to estimate traversability during the mission. We instead use a more expressive occupancy input representation resolving this failure mode.

C. Voxel-based Deep Learning

Predicting traversability from 3D voxel-based occupancy data requires computational efficient methods mitigating the underlying cubic complexity by exploiting sparsity. Wang et

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 14:53:34 UTC from IEEE Xplore. Restrictions apply.

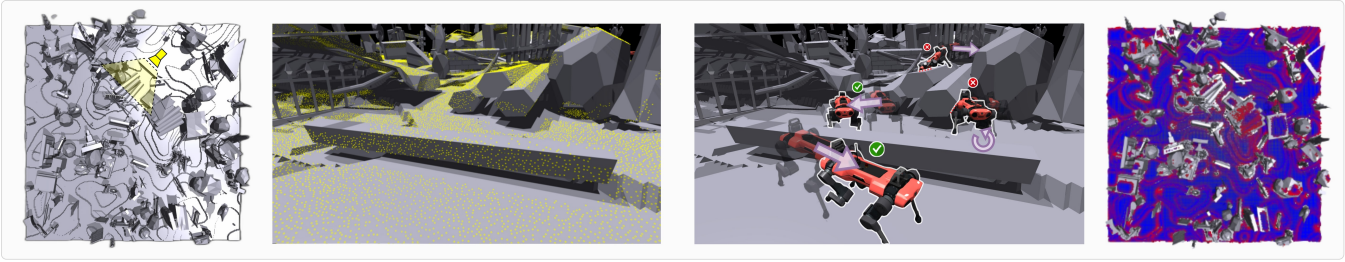


Fig. 2. Collecting traversability information in simulation (left to right): Birds-eye-view of randomly generated terrain mesh; Zoomed-in view of collision-free starting points in yellow; Multiple simulated ANYmal robots executing motion commands; Simulated traversability estimate color-coded.

al. [26] develop a neural network to operate on an octree volumetric data structure for shape analysis, restraining computations on occupied voxel octants. Sparse 3D CNNs restrain computation to occupied voxels [7]. These networks have been generalized to higher dimensions by Choy et al. [3]. We adopt the sparse encoder-decoder network of Gwak et al. [9] for semantic segmentation to predict traversability. The network utilizes sparse generative convolutions and pruning with sequential up-sampling and pruning voxels in 3D.

III. APPROACH

A. Problem Definition:

Given an environment representation and the internal state of the robot we assess if the robot is capable to reside over a given terrain. The environment is represented by an occupancy voxel map $\mathbf{O}(x, y, z) \in \{0, 1\}$ with a resolution of d_{voxel}, where each voxel is either labeled as free(0) or occupied(1). The state of the robot $\mathbf{s}$ is composed of its position $\mathbf{p} = (x, y, z)^T$, heading α, and locomotion policy π_θ,

$$\mathbf{s} = (\mathbf{p}, \alpha, \pi_\theta)^T \tag{1}$$

To store the traversability information associated with a terrain we introduce the notion of a traversability cost tensor $\mathbf{T}$. The tensor stores the traversability cost at a spatial location, for a set of robot configurations and motion commands a. We simplify the robot configuration to solely include the heading parameter α and discard velocities, individual joint positions and base pitch and roll, given that the traversability assessment is less dependent on these fast varying variables under nominal operating conditions of the studied legged robot ANYmal. Including the robot configuration and motion command is motivated by the fact that traversing up or down a slope imposes varying risk and e.g. within tight corridors only a subset of actions can be executed collision-free. The traversability cost tensor allows to capture this information adequately and measures traversability in a range of 0 to 1, where 0 is untraversable and 1 risk-free traversable.

$$\mathbf{T}_{\pi_\theta}(x, y, z, \alpha_j, a_j) \in [0, 1] \tag{2}$$

Our objective is to find the function f_{trav} which predicts each element within the traversability cost tensor $\mathbf{T}$, given the occupancy map representation $\mathbf{O} \in \mathcal{O}$ and state of the robot $\mathbf{s} \in \mathcal{S}$.

$$\begin{aligned} \mathbf{T}_{\pi_\theta}(x, y, z, \alpha_i, a_j) &= f_{trav}(\mathbf{s}, \mathbf{O}) \\ f_{trav} : \quad \mathcal{S} \times \mathcal{O} &\rightarrow \mathbb{R}_{[0,1]} \end{aligned} \tag{3}$$

B. Simulating Traversability:

The traversability cost is evaluated by simulating locomotion over randomly generated terrain in NVIDIA's Isaac-Gym [16]. To achieve accurate sim-to-real transfer the robot is controlled by the same locomotion policy later deployed on the real robot. In this work, the traversability cost is defined as the success rate of reaching the desired goal pose, which is of main interest for autonomous navigation. The goal pose is determined by the motion command with respect to the robots starting pose. Evaluation of multiple trials per motion command under domain randomization results in a robust geometric traversability risk assessment. We introduce the terrain generation procedure in (Sec. III-B.1), followed by the traversability data collection in (Sec. III-B.2).

1) Terrain Generation: We aim to generate a large and diverse terrain dataset covering the domain of terrains that the robot may encounter during deployment. Acquisition of a large amount of experience has shown to increase generalization performance for various tasks, including locomotion [18] and traversability estimation [30]. The ground structure of the terrain map is generated using Perlin noise with either a smooth surface or height-varying steps, following the commonly used terrain generation for training locomotion policies [18], [23]. To generate a complex environment with a variety of static obstacles, multiple objects with a diameter between $0.1\,\mathrm{m}$ and $6\,\mathrm{m}$ are randomly spawned in the map. These objects were originally generated by human artists to create 3D worlds for computer games. In total between 300 and 1000 objects are spawned with a random location and orientation. The spawning probability of each object is inversely scaled with the diameter of its axis-aligned bounding box leading to more smaller objects. Each spawned object's size is randomly scaled with a factor of 0.5 and 1.5. With a probability of $90\,\%$ the spawned object is aligned with the ground surface and otherwise spawned uniformly sampled between $0\,\mathrm{m}$ and $3\,\mathrm{m}$ above the ground resulting in overhanging obstacles. One example of a $32\,\mathrm{m} \times 32\,\mathrm{m}$ terrain patch is shown in Figure 2.

2) Traversability Data Collection: An initial set of robot starting poses is evaluated by uniformly sampling points on the generated terrain mesh. We evaluate if the robot can be spawned collision-free with nominal joint configuration above the sampled point. This is conducted for all headings α_k sampled with 10° resolution, resulting in a total of 36 possible start headings per point. The robots' pitch and roll

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 14:53:34 UTC from IEEE Xplore. Restrictions apply.

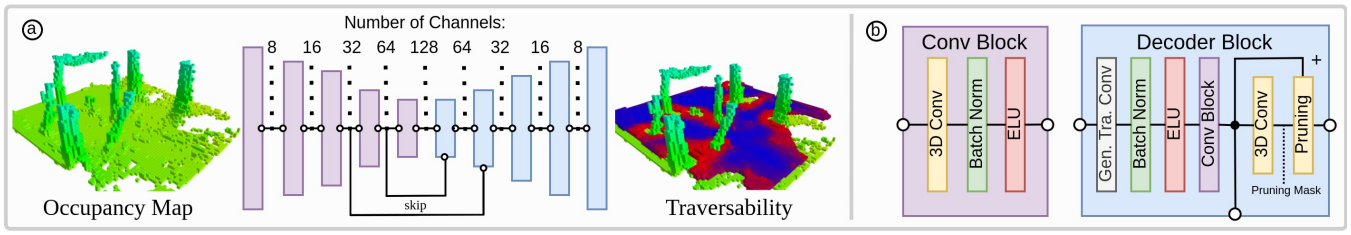


Fig. 3. (a) Simplified Sparse Encoder-Decoder 3D CNN Architecture, (b) Block Structure (for further detail refer to Section III-C.2 and [9])

base orientation are aligned with the underlying mesh. The base height is adjusted such that all feet are in proximity to the mesh surface. Only a single admissible robot starting pose for each heading is further evaluated per voxel with the same spatial resolution as the traversability cost tensor $\mathbf{T}$. Finally, to verify that the start position is safe for the robot to be initialized, $2\,\mathrm{s}$ of standing is simulated within the physics simulator with the current locomotion policy. If the robot is capable to remain at the same position successfully without collisions, the pose is labeled as a valid starting pose. Determining good starting poses is essential to reduce failure during traversability data collection due to a bad initialization of the robot. In addition, data collection is only performed for a reduced subset of initial starting poses in order to reduce the compute requirements.

For each of the valid spawn pose, $n_{total} = 10$ trials of each motion command a_i are executed. In particular, 4 translation and 2 rotation motions are performed. The translation distance is set to $40\,\mathrm{cm}$ along four cardinal directions (forward, backward, left, right) and the rotation is given by a $\pm 45°$ yaw. The selected motion commands determine the relative pose offset with respect to the starting pose. During execution the current pose offset of the robot with respect to the goal is used to compute the velocity command tracked by the locomotion policy. Four different trajectories with start (transparent) and goal poses (dense) with purple arrows indicating the motion command are shown in Figure 2. Each attempt results in a robot trajectory, which is evaluated for success based on foot contacts, base orientation, environment collision, time to reach the goal and distance to the goal. A maximum execution time of $4\,\mathrm{s}$ is set for each attempt. In Figure 2 the forward and side-ward moving robots reach their goal successfully (green check mark), whereas the other robots fail to execute the desired motion command given a collision with the environment (red cross). The overall traversability cost is given by the number of successful attempts n_{suc} divided by total attempts n_{total}.

We choose to use small pose offsets, such that a motion command can be correlated with the terrain surrounding the starting pose. Therefore the outcome of each trajectory is solely associated with the single starting entry of the traversability cost tensor, which is indexed by the starting robot position. Evaluating longer trajectories leads to the problem of determining which terrain part is responsible for the failed execution. Additionally, it is possible to increase the safety limits by adjusting the size of the used collision bodies. For each attempt the robot base mass and friction coefficient are randomized. The procedure to evaluate a trajectory on success can be further adapted depending on the application, which may include time, number of steps, or the robots' configuration. A sample of the collected terrain traversability cost is color-coded in Figure 2, where blue indicates risk-free traversable and red traversable under high risk.

C. Sparse 3D Traversability Prediction Network

1) Input: We train a sparse 3D CNN in a supervised manner to predict traversability information in real-time. Offline, for each terrain patch, the mesh is converted to a voxel occupancy map with a resolution of $10\,\mathrm{cm}$ using ray tracing. Every voxel containing a triangle is labeled as occupied. The occupancy map generated during the mission provides fused information of all past observations while functioning as an abstraction layer. The previously computed robot starting poses are used to crop a robot-centric training example from the larger terrain patch. The robot's base position is at the center of the extracted volume. The volume is rotated to align with robot's base yaw, while the z-axis is gravity aligned. This allows to implicitly provide the network the robot's pose, while the inclination of the terrain remains unchanged. We choose to use a window size of $8\,\mathrm{m} \times 8\,\mathrm{m} \times 4\,\mathrm{m}$ or respectively a volume size of $80 \times 80 \times 40$ voxels at a $10\,\mathrm{cm}$ resolution. The sparse 3D CNN is implemented using the Minkowski Engine [3].

2) Network: We adapt the generative sparse detection network developed for semantic segmentation [9]. The network consists of an encoder-decoder structure with skip connections between the respective feature maps and is illustrated in Figure 3. Each encoder block applies a sparse 3D convolution (stride=2, kernel=4), batch normalization, and exponential linear unit (ELU) as an activation function, which down-samples the spatial resolution by a factor of 2. The encoder generates feature maps with [8, 16, 32, 64, 128] channels. In contrast to the original network architecture skip connections are only applied in the two lower layers of the architecture. The intuition behind the choice of reducing the number of skip connections is discussed in Section IV-A. The decoder spatially up-samples the feature maps to the original resolution using generative transposed convolutional layers. At each decoder level a pruning mask is predicted by a sparse convolutional layer (stride=1, kernel=1) to increase the sparsity and reduce the computation needed. Only non-zero elements are propagated to the following layers. Furthermore, we tested applying our network in three

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 14:53:34 UTC from IEEE Xplore. Restrictions apply.

different output configurations, providing different levels of information that can be used for path planning purposes later. In the simplest case only the *total risk* is assessed, where the commands and robot configuration are marginalized within the traversability cost tensor resulting in a $c = 1$ dimensional target. The *orientation risk* when marginalizing over the motion commands results in a $c = 18$ dimensional traversability score with a $10°$ resolution when accounting for the symmetry of the robot. Lastly, the *c-bin motion direction risk* for which the relative motion direction of the applied action-heading combination, is used to bin them in c-equally sized motion direction bins.

3) Training: The network is trained to minimize the reconstruction and traversability estimation loss. The reconstruction loss is evaluated per decoder layer and is given by the weighted binary cross-entropy between predicted pruning mask and the respectively down-sampled target traversability cost tensor. The traversability loss is given by the mean squared error between the predicted and target traversability score. Here, the traversability loss is only assessed for correctly instantiated (true positive) points. ADAM with weight decay is used to optimize the network for 100.000 steps with a batch size of 8. The learning rate is scheduled according to the *1cycle* learning rate with a target learning rate of 0.001 [24]. During training at each network layer the correct-pruning mask is enforced for training stability.

The occupancy map created based on the full available mesh data does not resemble the data sensed by a robot during its mission. Surfaces are mostly observed from limited viewpoints, under sensor noise and odometry errors, which degrade the quality of the occupancy map generated online during a mission. We increase the robustness of the network using data augmentation. Firstly, only voxels are marked as traversable if a continuous traversable path exists, connecting the voxel to the starting robot position at the center. This is done by applying a flood-fill algorithm. The designed data augmentation exploits two underlying assumptions. Given the robots' size the traversability score is independent of the occupancy state of a voxel further away than $1\,\mathrm{m}$ from it. Additionally, the traversability cost prediction should be invariant to minor surface imperfections, which commonly arise due to discretization effects when generating occupancy maps. These two assumptions allow us to randomly drop occupied voxels and add surface imperfections by randomly spawning voxels within the proximity of occupied voxels.

IV. EXPERIMENTS

A. Training Results

To train our proposed network, we created 20 training and 10 validation terrain patches with a size of $32\,\mathrm{m}$ x $32\,\mathrm{m}$ following the procedure explained in Section III-A. Separate object asset datasets are used for generating the training and validation terrain. These objects were generated by professional artists and are freely available on the Unity Asset Store. We simulate 4096 robots to collect traversability information in parallel facilitating an accumulated real-time speed up factor of 400. Per patch 2500 minutes wall time

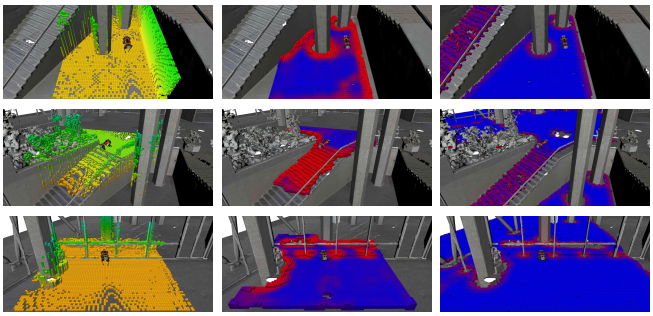


Fig. 4. ETH HPH building test dataset (left to right): Local Voxblox occupancy map, height is color-coded; Predicted *total risk* traversability cost for local occupancy map (blue risk-free traversable, red traversable under high risk); Simulated *total risk* traversability cost for full map-patch, (blue risk-free traversable, red traversable under high risk);

TABLE I

DATASETS OVERVIEW: ASSETS REEFERS TO THE NUMBER OF ASSETS GATHERED. INSTANCES IS THE RANGE OF SPAWNED ASSETS PER PATCH.

	Patches	Size [m]	Assets	Instances	Real
Train	20	32x32x16	300	300-1000	✗
Val	10	32x32x16	200	300-1000	✗
Test	5	32x32x16	-	-	✓

are spent collecting traversability using an RTX2080TI and 18 CPU cores of a AMD EPYC 7763. In total 57 years of real-time equivalent traversability information was gathered.

The test dataset is generated by applying the same traversability simulation procedure for two real-world meshes consisting of an underground mine in Switzerland (1 patch) and a multi-floor university building at the ETH Zurich (4 patches) shown in Figure 4. The mine mesh was collected during a preparatory run of team CERBERUS [25] for the DARPA Subterranean Challenge. All meshes were captured using a Leica RTC360 laser scanner. While the training and validation datasets are drawn from the same distribution, the test dataset allows assessing performance on real-world data. In the absence of ground truth, the generated test dataset allows for assessing the network's capability to reproduce the simulated traversability cost from the limited geometrical information provided by the occupancy representation. In Section IV-B we experimentally validate how this traversability estimate transfers to the real-world and can be used for path planning. In Table I we provide an overview of the generated datasets.

The root-mean-squared error (RMSE) between the predicted traversability $\hat{y}$ and simulated traversability score y is used to evaluate the performance. Not instantiated voxels are assumed to have a traversability of zero, and hence true negative (TN) voxels are not included when calculating the RMSE metric.

$$L_{RMSE} = \sqrt{\frac{1}{N} \sum_{TP,FN,FP} (y - \hat{y})^2} \qquad (4)$$
$$N = |TP| + |FN| + |FP|$$

During the training of the network, multiple hyper-parameters, including the learning rate, model capacity

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 14:53:34 UTC from IEEE Xplore. Restrictions apply.

TABLE II

RMSE REPORTED FOR DIFFERENT NETWORK ARCHITECTURES **M**, OUTPUT CHANNELS **C**, AND DATASETS (TRAIN, VALIDATION, TEST)

	$c = 1$			$c = 4$			$c = 18$		
	Tr	Val	Test	Tr	Val	Test	Tr	Val	Test
M1	0.124	0.131	0.101	0.232	0.236	0.162	0.450	0.465	0.385
M2	0.119	0.126	0.097	0.221	0.226	0.155	0.438	0.450	0.379

(number of channels), loss function hyper-parameters, data augmentations, and model architecture, were validated. We performed experiments for altering the number of skip-connections, which allow information forwarding between the encoder and decoder layers. Full skip-connectivity results in a large number of false positively classified voxels and overall worse performance (M1, Table II). Intuitively this can be explained by the fact that the decoder has to correctly predict the pruning mask for all forwarded features by the encoder. Only using skip connections in the two lower layers resulted in the lowest RMSE (M2, Table II). Furthermore, with the increase of task complexity (the number of output channels), the RMSE increases for all network architectures. This increase can be attributed to two reasons. First, the supervision signal includes more noise for a more complex task given that fewer robot trajectories are accumulated within a traversability estimate. Second, the model complexity is kept constant and a single pruning mask is predicted for all channels within the decoder.

Additionally, it can be noted that the performance on the test data is better than training data. We suspect that the real world data overall is an easier objective to estimate than the randomly generated terrain. This observation also demonstrates that the network is not overfitting to the training dataset. In Figure IV-A we deploy ANYmal, simulated in Gazebo, on the mesh of the university building at the ETH Zurich. We simulate the LiDAR sensors and use their measurements to build a robot-centric occupancy map (left). The network prediction (middle) and simulated traversability cost for the full map are color-coded. The trained network is conservative on the stairs. In addition, the ground truth traversability estimate on the stairs is noisy but consistent over the full staircase.

B. Real-World Deployment

We deployed ANYmal within various environments to verify the successful sim-to-real transfer. The robot is equipped with a front and back-facing RoboSense RS-BPearl LiDAR sensors. First, we illustrate the advantage of using a more expressive occupancy map representation and compare it to an elevation mapping-based approach. Second, we deploy the robot within a confined space to showcase the effectiveness of action aware traversability risk assessment. Last, we apply the network to data collected within a forest and an underground mine to showcase the effectiveness of our method for planning in complex real-world scenarios.

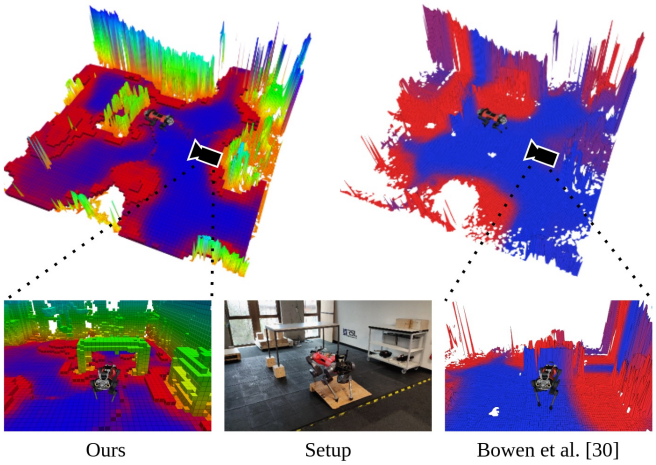


Fig. 5. Comparison of our network to an elevation mapping based traversability estimation approach. Our approach correctly labels the area underneath the desk as traversable. Elevation mapping based approach [30] is unable to produce a correct label for this area due to the corrupted elevation map.

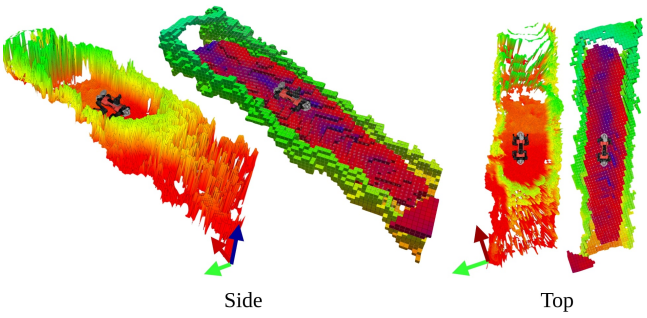


Fig. 6. Compromised elevation map in inclined low-ceiling mine shaft. Traversability prediction using the occupancy map. The ceiling is manually removed for visualization.

1) Overhanging Obstacles: This experiment is designed to highlight the need for using a more expressive 3D volumetric environment representation than the typically used 2.5D elevation maps for traversability estimation, especially in the presence of overhanging obstacles. Figure 5 shows the experimental setup with an elevated desk placed at the center of the room, allowing the ANYmal to pass underneath the desk without a special motion or gait. The elevation and occupancy map are generated using the same LiDAR measurements and robot pose estimates provided by Comp-SLAM [13]. The elevation map is generated with a spatial resolution of $5\,\mathrm{cm}$ on a Nvidia Jetson Xavier AGX [17]. The occupancy map is generated using VoxBlox [20] with a spatial resolution of $10\,\mathrm{cm}$ on the CPU. The occupancy map adequately captures the free space underneath the desk, while the elevation map of the same scene is unable to represent the area under the desk as free given that points on the tabletop are integrated. We compare our work to the traversability estimation approach proposed in [30], which uses an elevation map as an input. This network is trained on simulated data for ANYmal using a different data generation procedure. We use the estimated risk output of [30] to conduct a qualitative comparison solely. No quantitative

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 14:53:34 UTC from IEEE Xplore. Restrictions apply.

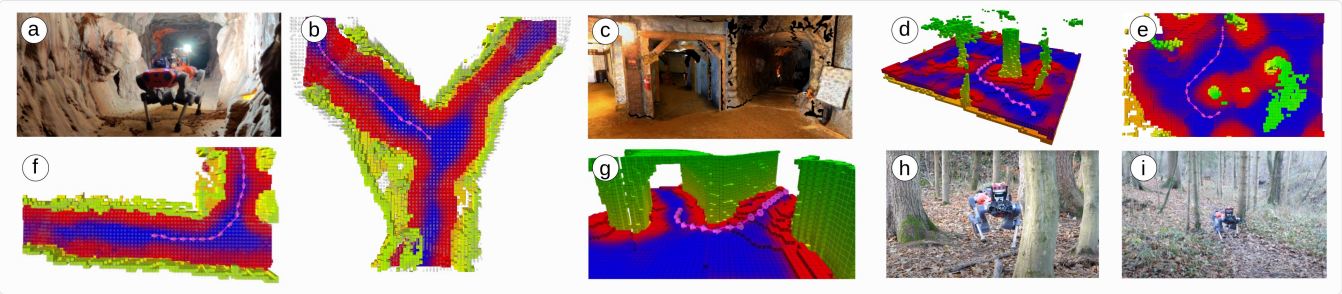


Fig. 7. Path planning in complex natural environment based on traversability estimate. Planned path is illustrated in purple. Forest (d, e, h, i), Cave (a, b, c, f, g). In the cave images the ceiling voxels are manually removed for illustration purposes.

results are presented given that both networks are trained on different data, and no ground truth information of the real traversability score can be obtained. The network of [30] is queried for each cell within the elevation map to assess the risk for 8 different headings when moving forward. In total 204.800 start-goal pairs per elevation map are evaluated and the average risk is color-coded per cell in Figure 5. Red indicates untraversable terrain and blue risk-free traversable terrain. Given the corrupted input representation, the area underneath the desk is classified as non-traversable, which prohibits planning underneath the desk. As a comparison, the output of our *total risk* network configuration is shown on the left in Figure 5 for the same scene. Here it is essential to distinguish between the color-coding used. In our work red implies traversable with a high risk ([30] untraversable) and blue risk-free traversable ([30] risk-free traversable). Our network predicts a traversability risk more adequately for the presented scene. Close to obstacles, the risk increases, and the occupied space is labeled as untraversable. Additionally, in Figure 6 we illustrate a scenario of ANYmal traversing in an inclined mine shaft with a low ceiling. Due to low ceiling clearance the elevation map gets corrupted as depth measurements on the ceiling are integrated. Our network, on the other hand, utilizing a 3D occupancy map can adequately predict the traversability cost. The prediction only includes minor errors (bottom left, predicting traversable region on top of the mine shaft), which can optionally be filtered out using a flood-fill algorithm. Here it is important to mention that scenarios like long steep inclined tunnels are not specifically covered within the training data.

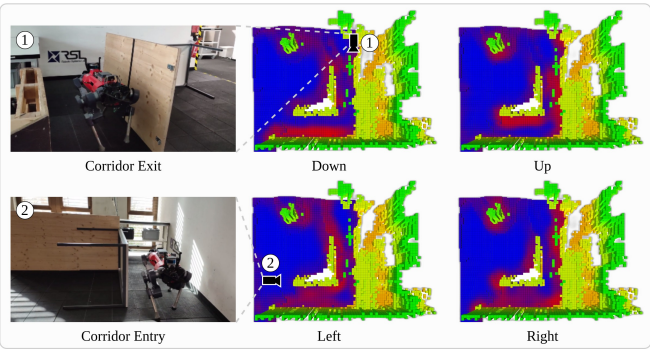


Fig. 8. Deployment of ANYmal within a confined corridor. Individual output channels for *4-bin motion direction risk* estimation.

2) Confined Spaces: In the following experiment, we deploy ANYmal within a confined space in an L-shaped corridor. The experimental setup is visualized from two viewpoints in Figure 8. The output of the network is illustrated for the *4-bin motion direction risk* configuration. The bins can be associated with moving up [315°,45°], right [45°,135°], down [135°,225°] and left [225°,315°]. A higher risk is associated with actions that lead to a collision. Within the corridor entry, the cost of moving down and up strongly depends on the robot's position. This additional information can be helpful for safe path planning and allows to limit or rank motion commands at a spatial location within a terrain according to the excepted risk. During path planning this additional information can guide the search and may improve the planning performance in terms of risk or planning time.

3) Path Planning in Complex and Natural Environments: To illustrate the real-world application of our proposed method, we deployed it on data gathered within a forest and the data from the DARPA subterranean challenges final run of team CERBERUS. We use the *total risk* network configuration to predict the traversability based on the generated occupancy map. Within the forest and underground cave, no non-rigid obstacles as e.g. high-grass or bushes, are present. Therefore the generated occupancy map representation correctly reflects the support surfaces.

To showcase the application of our traversability estimation network we implement a simple local planner capable to find a safe path between the current robot position at the center and an externally provided target position. Given the predicted traversability cost a directional graph is generated. Each traversable voxel is a node within the graph connected with its 26-neighborhood by an edge with a cost of c_{s-g}:

$$c_{s-g} = \|\mathbf{p}_g - \mathbf{p}_s\|_2 + \lambda(1 - \hat{\mathbf{T}}(\mathbf{p}_g)), \tag{5}$$

where $\mathbf{p}_{s/g}$ denotes the position of the start and goal node. The hyper-parameter λ trades off the length and total risk of the path. By increasing λ, lower total risks paths are preferred at the cost of a geometrical longer path between start and goal node. The Dijkstra algorithm is used to find the shortest path between the current robot position and the provided target position. The resulting path for the forest and cave environment with $\lambda = 0.1$ is illustrated in purple for all 5 example scenes in Figure 7. Within the forest, the generated path traverses with a safety margin around all nearby trees. In the cave environment, the generated path traverses along

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 14:53:34 UTC from IEEE Xplore. Restrictions apply.

the center of the corridor, maximizing the distance between both walls. Finally, we used the dataset collected within the forest to assess the average inference time of the network on a Nvidia Jetson Xavier AGX. The inference time depends on the number of occupied voxels given that computation is only performed in a sparse manner using the Minkowski Engine and terrain geometry. On average, the network runs at $16.5\,\mathrm{hz}\pm1.2\,\mathrm{hz}$ processing 12181 ± 1611 input voxels. The inference time is measured over 100 selected key-frames in the forest dataset and the number of occupied voxels may change depending on the environment. Given the update rate of each LiDAR is $10\,\mathrm{hz}$ the network is real-time capable.

V. CONCLUSION

In this work we showed that the proposed sparse 3D convolution neural network trained on traversability data collected in simulation on randomly generated terrains can accurately predict the traversability for unseen terrains using a fully geometric occupancy representations as input. Therefore the network trained exclusively on simulated data generalizes successfully to the real world. It successfully handles overhanging obstacles, outperforming elevation map based approaches in these scenarios, while being real-time capable. While we showed in multiple experiment successful sim-2-real transfers a more exhaustive study design of the sim-2-real transfer is needed. Not only a sim-2-real transfer of the actual traversability estimation network is needed but in addition the sim-2-real transfer of the deployed locomotion policy. For future work we will investigate integration of visual semantic information to assess friction coefficients or the quality of individual footholds. Additionally, no temporal consistency is enforced between individual predictions in the current implementation. This leads to changing predictions for individual voxels over time. Expanding the created robot-specific traversability estimation module to different robot categories will allow to develop a unified planning system operating across robot types without requiring robot-specific heuristics. Finally, exploiting the generated traversability estimates for a variety of downstream tasks, including active switching between locomotion policies or gait patterns, local and global path planning, and exploration is promising.

REFERENCES

[1] A. Bouman and et. al, "Autonomous spot: Long-range autonomous exploration of extreme environments with legged locomotion," *CoRR*, vol. abs/2010.09259, 2020.

[2] R. O. Chavez-Garcia, J. Guzzi, L. M. Gambardella, and A. Giusti, "Learning ground traversability from simulations," *CoRR*, vol. abs/1709.05368, 2017.

[3] C. B. Choy, J. Gwak, and S. Savarese, "4d spatio-temporal convnets: Minkowski convolutional neural networks," *CoRR*, vol. abs/1904.08755, 2019.

[4] D. D. Fan and et. al, "Step: Stochastic traversability evaluation and planning for risk-aware off-road navigation," *Robotics: Science and Systems XVII*, 2021.

[5] P. Fankhauser and et. al, "Robot-centric elevation mapping with uncertainty estimates," in *International Conference on Climbing and Walking Robots (CLAWAR)*, 2014.

[6] J. Frey, H. Blum, F. Milano, R. Siegwart, and C. Cadena, "Continual learning of semantic segmentation using complementary 2d-3d data representations," *CoRR*, vol. abs/2111.02156, 2021.

[7] B. Graham, M. Engelcke, and L. van der Maaten, "3d semantic segmentation with submanifold sparse convolutional networks," *CoRR*, vol. abs/1711.10275, 2017.

[8] J. Guzzi, R. O. Chavez-Garcia, M. Nava, L. M. Gambardella, and A. Giusti, "Path planning with local motion estimations," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2586–2593, 2020.

[9] J. Gwak, C. B. Choy, and S. Savarese, "Generative sparse detection networks for 3d single-shot object detection," in *European conference on computer vision*, 2020.

[10] L. Han, F. Gao, B. Zhou, and S. Shen, "FIESTA: fast incremental euclidean distance fields for online motion planning of aerial robots," *CoRR*, vol. abs/1903.02144, 2019.

[11] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, 2013.

[12] L. D. Jackel and et. al, "The darpa lagr program: Goals, challenges, methodology, and phase i results," *Journal of Field Robotics*, vol. 23, no. 11-12, pp. 945–973, 2006.

[13] S. Khattak and et al., "Complementary multi–modal sensor fusion for resilient robot pose estimation in subterranean environments," in *IEEE International Conference on Unmanned Aircraft Systems*, 2020.

[14] P. Krüsi and et al., "Driving on point clouds: Motion planning, trajectory optimization, and terrain assessment in generic nonplanar environments," *Journal of Field Robotics*, vol. 34, pp. 940–984, 2017.

[15] M. Kulkarni and et. al, "Autonomous teamed exploration of subterranean environments using legged and aerial robots," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.

[16] V. Makoviychuk *et al.*, "Isaac gym: High performance GPU based physics simulation for robot learning," in *Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

[17] T. Miki and et al., "Elevation mapping for locomotion and navigation using gpu," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2022), under review*, 2022.

[18] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.

[19] K. Museth, "Vdb: High-resolution sparse volumes with dynamic topology," *ACM Trans. Graph.*, vol. 32, no. 3, 2013.

[20] H. Oleynikova and et. al, "Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[21] K. Otsu and et. al, "Autonomous terrain classification with co- and self-training approach," *IEEE Robotics and Automation Letters*, vol. 1, no. 2, pp. 814–819, 2016.

[22] P. Papadakis, "Terrain traversability analysis methods for unmanned ground vehicles: A survey," *Engineering Applications of Artificial Intelligence*, vol. 26, no. 4, pp. 1373–1385, 2013.

[23] N. Rudin and et. al, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proceedings of the 5th Conference on Robot Learning*, 2022, pp. 91–100.

[24] L. N. Smith and N. Topin, "Super-convergence: Very fast training of residual networks using large learning rates," *CoRR*, vol. abs/1708.07120, 2017.

[25] M. Tranzatto and et. al, "Cerberus: Autonomous legged and aerial robotic exploration in the tunnel and urban circuits of the darpa subterranean challenge," 2022.

[26] P. Wang, Y. Liu, Y. Guo, C. Sun, and X. Tong, "O-CNN: octree-based convolutional neural networks for 3d shape analysis," *CoRR*, vol. abs/1712.01537, 2017.

[27] L. Wellhausen and et. al, "Where should i walk? predicting terrain properties from images via self-supervised learning," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1509–1516, 2019.

[28] L. Wellhausen and M. Hutter, "Rough terrain navigation for legged robots using reachability planning and template learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021.

[29] M. Wermelinger and et. al, "Navigation planning for legged robots in challenging terrain," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 1184–1189.

[30] B. Yang and et. al, "Real-time optimal navigation planning using learned motion costs," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 9283–9289.

Authorized licensed use limited to: Universita degli Studi di Genova. Downloaded on April 07,2026 at 14:53:34 UTC from IEEE Xplore. Restrictions apply.